
MATH 135B - Project 1 - Joseph Benitez

Table of Contents

Approximation of Pi using Numerical Integration	1
--- Setup ---	1
--- PART 1: Composite Rules (Midpoint, Trapezoid, Simpson) ---	1
--- PART 2: Romberg Integration ---	3
--- PART 3: Adaptive Trapezoid Method ---	4
--- Local Functions ---	4

Approximation of Pi using Numerical Integration

```
clear; clc; close all;
```

--- Setup ---

Define the integrand function $f(x) = 4 / (1 + x^2)$

```
FoX = @(x) 4 ./ (1 + x.^2);
```

```
% Exact value of pi for error calculation
```

```
exact_pi = pi;
```

--- PART 1: Composite Rules (Midpoint, Trapezoid, Simpson) ---

We use powers of 2 for n to ensure n is even (required for Simpson) and to observe convergence clearly.

```
n_powers = 1:10;
```

```
n_values = 2.^n_powers;
```

```
% Pre-allocate error arrays
```

```
error_Mid = zeros(size(n_values));
```

```
error_Trap = zeros(size(n_values));
```

```
error_Simp = zeros(size(n_values));
```

```
fprintf('Part 1: Composite Rules Errors\n');
```

```
fprintf('%-6s %-15s %-15s %-15s\n', 'n', 'Err Midpoint', 'Err Trap', 'Err Simpson');
```

```
for k = 1:length(n_values)
```

```
    n = n_values(k);
```

```

a = 0; b = 1;
h = (b-a)/n;

% Generate nodes x0, x1, ..., xn
x = linspace(a, b, n+1);

% --- Midpoint Rule ---
x_mid = x(1:end-1) + h/2;
M_val = h * sum(FoX(x_mid));
error_Mid(k) = abs(M_val - exact_pi);

% --- Composite Trapezoid Rule ---
T_val = (h/2) * (FoX(x(1)) + 2*sum(FoX(x(2:end-1))) + FoX(x(end)));
error_Trap(k) = abs(T_val - exact_pi);

% --- Simpson's Rule ---
odds = x(2:2:end-1);
evens = x(3:2:end-2);
S_val = (h/3) * (FoX(x(1)) + 4*sum(FoX(odds)) + 2*sum(FoX(evens)) +
FoX(x(end)));
error_Simp(k) = abs(S_val - exact_pi);

% Print errors for this n
fprintf('%-6d %-15.4e %-15.4e %-15.4e\n', n, error_Mid(k), error_Trap(k),
error_Simp(k));
end

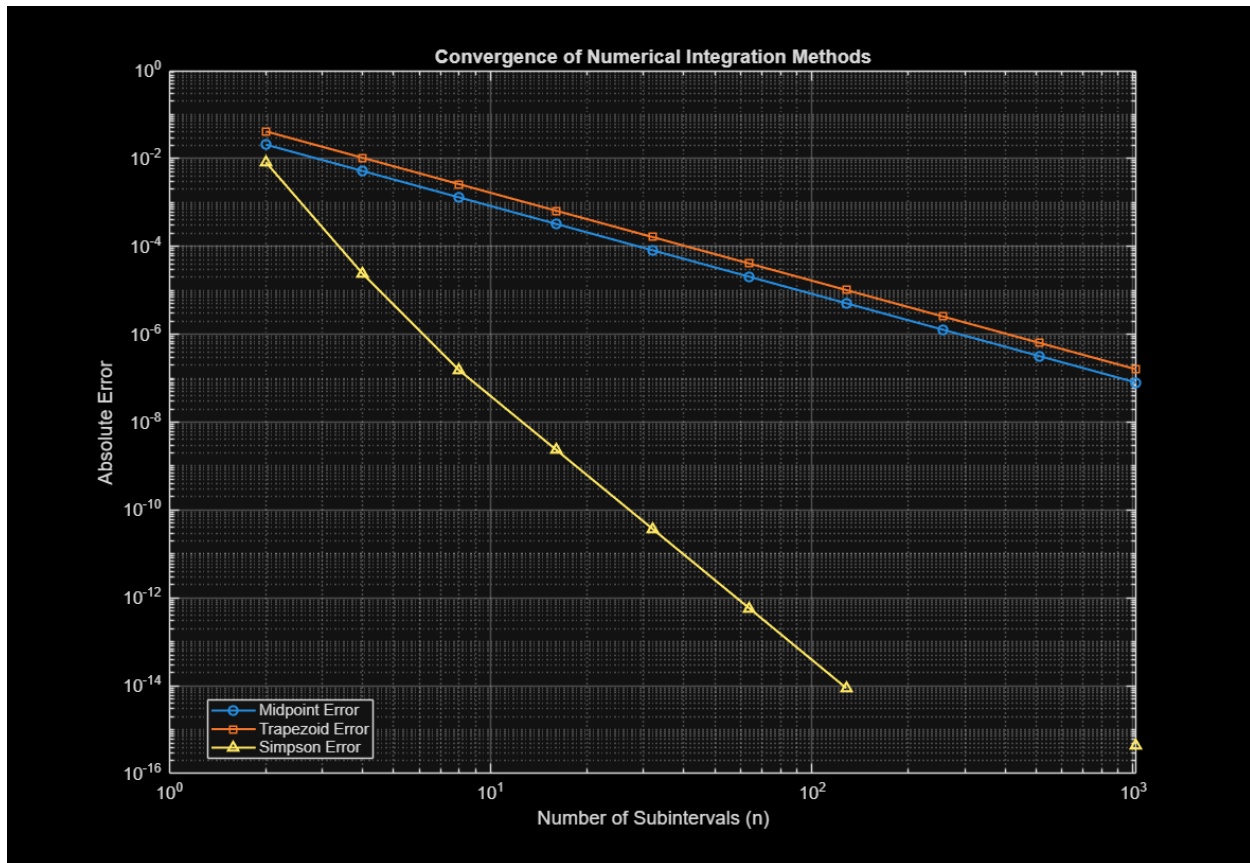
% --- Log-Log Plot ---
figure;
loglog(n_values, error_Mid, '-o', 'LineWidth', 1.5, 'DisplayName', 'Midpoint
Error');
hold on;
loglog(n_values, error_Trap, '-s', 'LineWidth', 1.5, 'DisplayName',
'Trapezoid Error');
loglog(n_values, error_Simp, '-^', 'LineWidth', 1.5, 'DisplayName', 'Simpson
Error');
hold off;
xlabel('Number of Subintervals (n)');
ylabel('Absolute Error');
title('Convergence of Numerical Integration Methods');
legend('Location', 'best');
grid on;
set(gca, 'FontSize', 12);

```

Part 1: Composite Rules Errors

<i>n</i>	<i>Err Midpoint</i>	<i>Err Trap</i>	<i>Err Simpson</i>
2	2.0760e-02	4.1593e-02	8.2593e-03
4	5.2079e-03	1.0416e-02	2.4026e-05
8	1.3021e-03	2.6042e-03	1.5113e-07
16	3.2552e-04	6.5104e-04	2.3650e-09
32	8.1380e-05	1.6276e-04	3.6957e-11
64	2.0345e-05	4.0690e-05	5.7687e-13

128	5.0863e-06	1.0173e-05	8.8818e-15
256	1.2716e-06	2.5431e-06	0.0000e+00
512	3.1789e-07	6.3578e-07	0.0000e+00
1024	7.9473e-08	1.5895e-07	4.4409e-16



--- PART 2: Romberg Integration ---

```
fprintf('\nPart 2: Romberg Integration Table R(j,k)\n');

% R table size (calculating up to R(5,5))
rows = 5;
R = zeros(rows, rows);

% Column 1: Trapezoid Rule for n = 1, 2, 4, 8, 16
for j = 1:rows
    n = 2^(j-1);
    h = (1-0)/n;
    x = linspace(0, 1, n+1);
    R(j, 1) = (h/2) * (FoX(x(1)) + 2*sum(FoX(x(2:end-1))) + FoX(x(end)));
end

% Compute columns 2 through rows using Richardson Extrapolation
for k = 2:rows
    for j = k:rows
        % Formula: R(j,k) = R(j,k-1) + [R(j,k-1) - R(j-1,k-1)] / (4^(k-1) - 1)
    end
end
```

```

        R(j, k) = R(j, k-1) + (R(j, k-1) - R(j-1, k-1)) / (4^(k-1) - 1);
    end
end

% Display the Lower Triangular Table
disp(R);
fprintf('Approximation R(5,5): %.16f\n', R(5,5));
fprintf('Exact Pi           : %.16f\n', exact_pi);
fprintf('Romberg Error       : %.4e\n', abs(R(5,5) - exact_pi));

```

Part 2: Romberg Integration Table $R(j,k)$

3.0000	0	0	0	0
3.1000	3.1333	0	0	0
3.1312	3.1416	3.1421	0	0
3.1390	3.1416	3.1416	3.1416	0
3.1409	3.1416	3.1416	3.1416	3.1416

Approximation $R(5,5)$: 3.1415926652777171
Exact π : 3.1415926535897931
Romberg Error : 1.1688e-08

--- PART 3: Adaptive Trapezoid Method ---

```

tol = 1e-8;
% Call the local function defined at the end of the script
adaptive_val = adaptive_trap_runner(FoX, 0, 1, tol);

fprintf('\nPart 3: Adaptive Trapezoid Method\n');
fprintf('Tolerance           : %.1e\n', tol);
fprintf('Approximation         : %.16f\n', adaptive_val);
fprintf('Absolute Error        : %.4e\n', abs(adaptive_val - exact_pi));
% Note: num_evals is tricky to track in recursion without global vars or
% passing counts,
% but the main result is the approximation.

```

--- Local Functions ---

```

function [I] = adaptive_trap_runner(f, a, b, tol)
    % Initial call helper to start recursion
    % We compute the first trapezoid step here
    I = adaptive_recursive(f, a, b, tol, f(a), f(b));
end

function I = adaptive_recursive(f, a, b, tol, fa, fb)
    % Inputs:
    % f: function handle
    % a, b: interval boundaries
    % tol: error tolerance
    % fa, fb: function values at boundaries (to avoid re-evaluating)

    % 1. One-interval Trapezoid (coarse)
    T1 = 0.5 * (b - a) * (fa + fb);

```

```
% 2. Two-interval Trapezoid (fine)
m = (a + b) / 2;
fm = f(m);
T2 = 0.5 * (T1 + (b - a) * fm); % Algebraic update for Trap rule

% 3. Error Estimate
% Using the 1/3 factor common in adaptive schemes (related to Simpson's)
error_est = (1/3) * abs(T2 - T1);

if error_est < tol
    % If error is small enough, accept T2 (or Richardson extrapolated
    value)
    % Returning T2 + (T2-T1)/3 is actually Simpson's rule (Adaptive
    Simpson),
    % but since the prompt asks for "Adaptive Trapezoid", we usually
    return T2.
    I = T2;
else
    % Split interval and recurse
    % Tolerance scales by half to maintain global error bound
    I = adaptive_recursive(f, a, m, tol/2, fa, fm) + ...
        adaptive_recursive(f, m, b, tol/2, fm, fb);
end
end
```

Part 3: Adaptive Trapezoid Method
Tolerance : 1.0e-08
Approximation : 3.1415926528563611
Absolute Error : 7.3343e-10

Published with MATLAB® R2025b